

《移动终端软件开发技术》

Mobile Application Development

5. Android数据存储

谢坤
计算机学院（国家示范性软件学院）

数据持久化

- 瞬时数据
 - 存储在内存当中
 - 有可能会因为程序关闭或其他原因导致内存被回收而丢失
- 持久数据
 - 保存在存储设备中
 - 即使在设备关机的情况下仍然不会丢失
- 持久化技术
 - 让数据在瞬时状态和持久状态之间进行转换的一种机制
 - 文件存储、SharedPreferences存储、数据库存储

1 文件存储

1.1 文件存储

- 直接使用文件系统保存数据
 - 基于Linux的文件系统
- 最基本的数据存储方式
 - 不做格式化处理
 - 适合存储简单的文本数据或二进制数据文件
 - 存储结构化数据需自定义解析机制

1.2 存储数据到文件

- `openFileOutput()`
 - 为写入数据做准备而打开文件
 - 如果指定的文件存在，直接打开文件准备写入数据
 - 如果指定的文件不存在，则创建一个新的文件
 - 两个参数：
 - 参数1：文件名称
 - `/data/data/<package name>/files/`
 - 参数2：操作模式
 - 返回值是`FileOutputStream`类型

1.2 存储数据到文件

```
1. fun save(inputText: String) {  
2.     try {  
3.         val output = openFileOutput("data", Context.MODE_PRIVATE)  
4.         val writer = BufferedWriter(OutputStreamWriter(output))  
5.         writer.use {  
6.             it.write(inputText)  
7.         }  
8.     } catch (e: IOException) {  
9.         e.printStackTrace()  
10.    }  
11. }
```

1.2 存储数据到文件

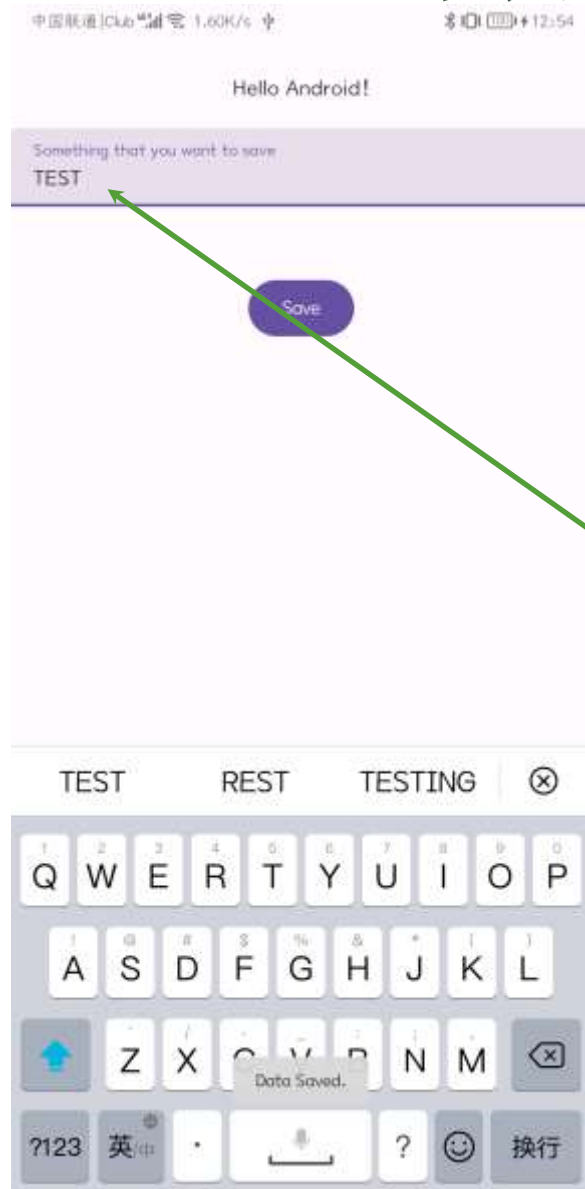
```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    var stringInput by remember { mutableStateOf("") }
    val context = LocalContext.current

    Column (.....){
        .....
        EditStringField(
            label = R.string.something_to_save,
            keyboardOptions = KeyboardOptions.Default.copy(
                keyboardType = KeyboardType.Text,
                imeAction = ImeAction.Done
            ),
            value = stringInput,
            onValueChanged = { stringInput = it },
            modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth(),
        )
        .....
        Button( onClick = { save(stringInput, context) } ) {
            Text(stringResource(R.string.button_save))
        }
    }
}
```

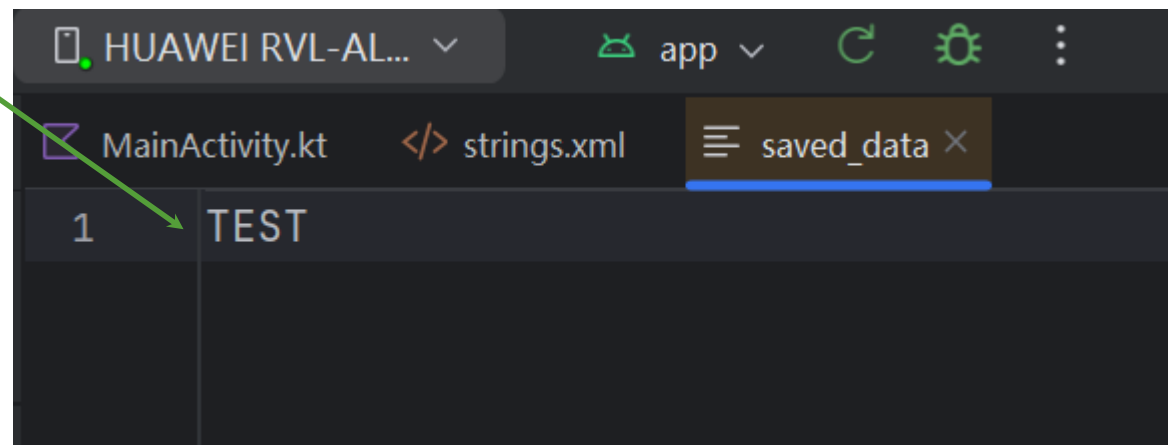
```
@Composable
fun EditStringField(
    @StringRes label: Int,
    keyboardOptions: KeyboardOptions,
    value: String,
    onValueChanged: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    TextField(
        value = value,
        singleLine = true,
        modifier = modifier,
        onValueChange = onValueChanged,
        label = { Text(stringResource(label)) },
        keyboardOptions = keyboardOptions
    )
}
```

```
private fun save(textInput: String, context: Context) {
    try {
        val output = context.openFileOutput("saved_data", Context.MODE_PRIVATE)
        val writer = BufferedWriter(OutputStreamWriter(output))
        writer.use {
            it.write(textInput)
        }
        Toast.makeText(context, "Data Saved.", Toast.LENGTH_LONG).show()
    } catch (e: IOException) {
        e.printStackTrace()
    }
}
```

1.2 存储数据到文件



>	com.errorsevendev.games.IB:	drwxrwx--x	2025-01-16 23:18	4 KB
>	com.evernote	drwxrwx--x	2025-01-16 23:18	4 KB
✓	com.example.cfilepersistence	drwxrwx--x	2025-01-16 23:18	4 KB
>	cache	drwxrws--x	2025-01-20 12:43	3.4 KB
>	code_cache	drwxrws--x	2025-01-20 12:43	3.4 KB
✓	files	drwxrwx--x	2025-01-20 12:49	3.4 KB
	profileInstalled	-rw-----	2025-01-20 12:49	24 B
	saved_data	-rw-rw----	2025-01-20 12:49	12 B
>	com.facebook.katana	drwxrwx--x	2025-01-16 23:18	4 KB
>	com.flyersoft.moonreaderp	drwxrwx--x	2025-01-16 23:18	4 KB



1.3 从文件读取数据

- `openFileInput()`
 - 为读取数据做准备而打开文件
 - 一个参数：
 - 文件名称，不允许包含描述路径的斜杠
 - 返回值：
 - `FileInputStream`对象

1.3 从文件读取数据

```
1. fun load(): String {  
2.     val content = StringBuilder()  
3.     try {  
4.         val input = openFileInput("data")  
5.         val reader = BufferedReader(InputStreamReader(input))  
6.         reader.use {  
7.             reader.forEachLine {  
8.                 content.append(it)  
9.             }  
10.        }  
11.    } catch (e: IOException) {  
12.        e.printStackTrace()  
13.    }  
14.    return content.toString()  
15. }
```

1.3 从文件读取数据

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    val context = LocalContext.current
    val oldString = load(context)
    var stringInput by remember { mutableStateOf(oldString) }

    Column (.....){.....}
}

private fun load(context: Context): String {
    val content = StringBuilder()
    try {
        val input = context.openFileInput("saved_data")
        val reader = BufferedReader(InputStreamReader(input))
        reader.use {
            reader.forEachLine {
                content.append(it)
            }
        }
        Toast.makeText(context, "Data Loaded.", Toast.LENGTH_LONG).show()
    } catch (e: FileNotFoundException) {
        return ""
    } catch (e: IOException) {
        return ""
    }
    return content.toString()
}
```

中国联通 | Club 32.0B/s

1:34

Hello Android!

Something that you want to save
Another Test

Save

Data Loaded.

2 SharedPreferences存储

2.1 简单存储

- SharedPreferences
 - 轻量级的数据保存方式
 - 调用其方法可实现对NVP (Name/Value Pair, 键/值对) 在Android的文件系统中的保存和读取
 - 完全屏蔽对文件系统的操作过程

2.2 存储数据到S.P.

- Context类中的getSharedPreferences()方法
 - 两个参数：
 - 第1个：用于指定SharedPreferences文件的名称
 - /data/data/<package name>/shared_prefs/
 - 第2个：用于指定操作模式
- Activity类中的getPreferences()方法
 - 一个参数：
 - 用于指定操作模式

2.2 存储数据到S.P.

- 利用SharedPreferences对象存储数据
 1. 调用edit()方法获取SharedPreferences.Editor对象
 2. 调用一系列put方法向Editor对象中添加数据
 3. 调用apply()方法提交添加的数据

2.2 存储数据到S.P.

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    val context = LocalContext.current
    Column (
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text( text = "Hello $name!" )
        Button(
            onClick = {
                val editor = context.getSharedPreferences("data", Context.MODE_PRIVATE).edit()
                editor.putString("name", "Tom")
                editor.putInt("age", 28)
                editor.putBoolean("married", false)
                editor.apply()
                Toast.makeText(context, "Data Saved.", Toast.LENGTH_LONG).show()
            }
        ) { Text(stringResource(R.string.button_save)) }
    }
}
```

Hello Android!

Save Data

Load Data

Data Saved.

2.2 存储数据到S.P.

```
> com.example.cfilepersistencetest drwxrwx--> 2025-01- 4 KB
✓ com.example.csharedpreferencetest drwxrwx--> 2025-01- 4 KB
  > cache drwxrws--x 2025-01- 3.4 KB
  > code_cache drwxrws--x 2025-01- 3.4 KB
  > files drwxrwx--> 2025-01- 3.4 KB
  ✓ shared_prefs drwxrwx--> 2025-01- 3.4 KB
    </> data.xml -rw-rw---- 2025-01- 186 B
> com.facebook.katana drwxrwx--> 2025-01- 4 KB
> com.flyersoft.moonreaderp drwxrwx--> 2025-01- 4 KB
```

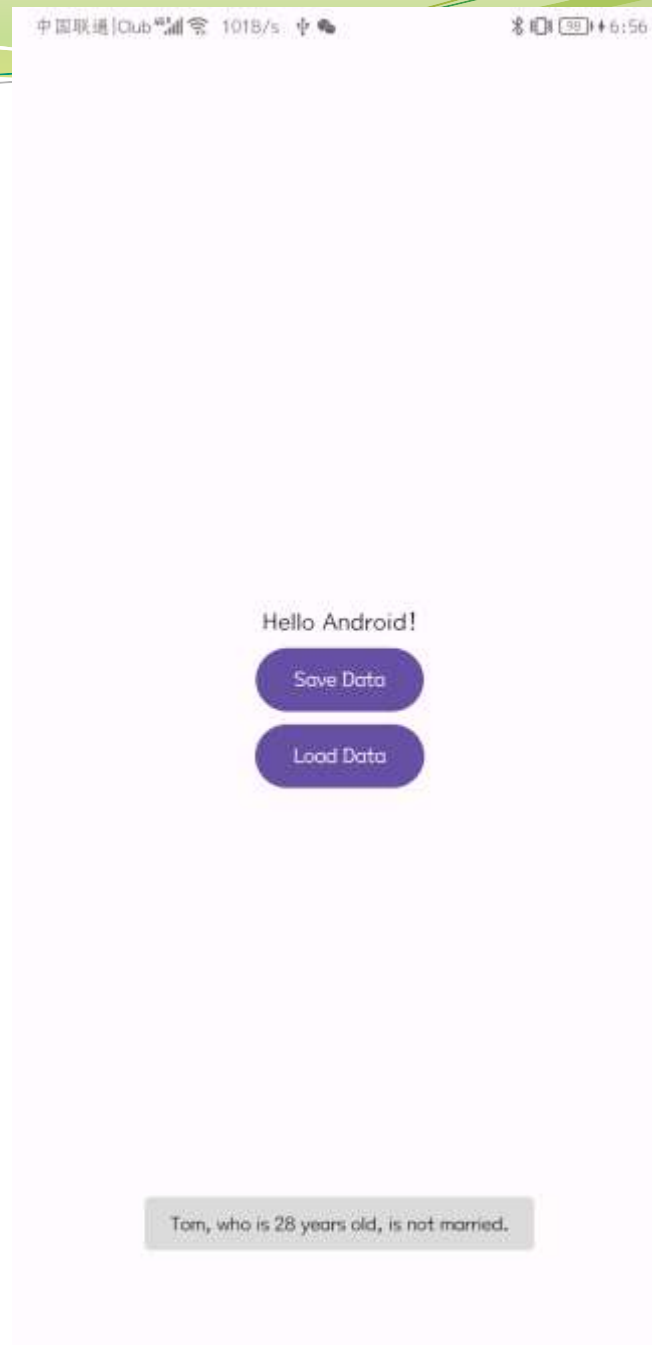
```
MainActivity.kt  </> data.xml ×  </> strings.xml
1  <?xml version='1.0' encoding='utf-8' standalone='yes' ?>
2  <map>
3      <string name="name">Tom</string>
4      <boolean name="married" value="false" />
5      <int name="age" value="28" />
6  </map>
```

2.3 从S.P.读取数据

- 从SharedPreferences文件读取数据
 - 调用一系列get方法读取数据
 - 两个参数
 - 第1个：键
 - 第2个：默认值

2.3 从S.P.读取数据

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    val context = LocalContext.current
    Column (.....) {
        .....
        Button(
            onClick = {
                val prefs = context.getSharedPreferences("data", Context.MODE_PRIVATE)
                val sName = prefs.getString("name", "Nobody")
                val sAge = prefs.getInt("age", 0)
                val sMarried = prefs.getBoolean("married", false)
                val isMarried = if (sMarried) "is" else "is not"
                Toast.makeText(
                    context,
                    "$sName, who is $sAge years old, $isMarried married.",
                    Toast.LENGTH_LONG
                ).show()
            }
        ) { Text(stringResource(R.string.button_load)) }
    }
}
```



3 SQLite数据库存储

3.1 SQLite数据库

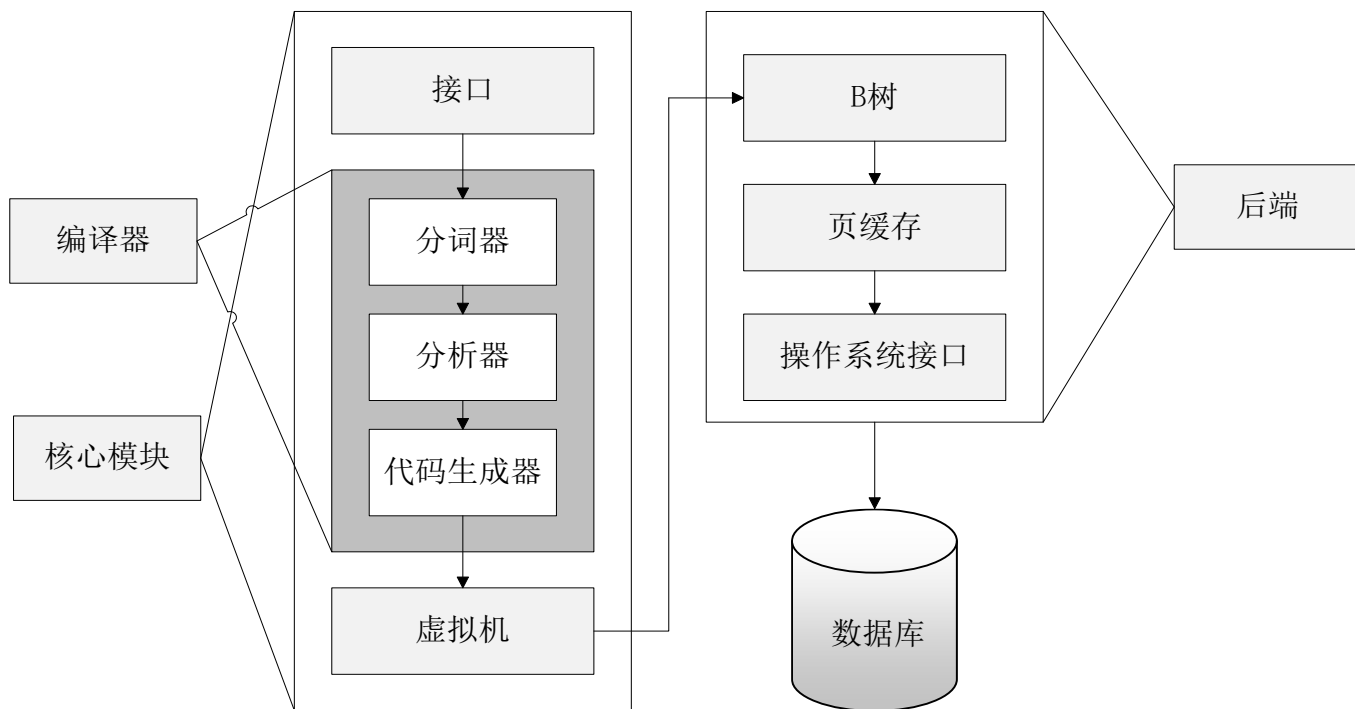
- SQLite数据库
 - 开源嵌入式关系数据库
 - 普通数据库的管理系统比较庞大和复杂，会占用较多系统资源
 - 轻量级数据库SQLite的特点
 - 更适合用于嵌入式系统
 - 占用资源少，运行高效可靠，可移植性强
 - 提供零配置（zero-configuration）运行模式

3.1 SQLite数据库

- SQLite数据库的优势
 - 嵌入到使用它的应用程序中
 - 提高运行效率
 - 屏蔽数据库使用和管理复杂性
 - 客户端和服务端在同一进程空间运行
 - 不需进行网络配置和管理
 - 减少网络调用所造成的额外开销
 - 移植性
 - 可运行在Windows、Linux、BSD、Mac OS和一些商用Unix系统上
 - 可运行在大量嵌入式操作系统上

3.1 SQLite数据库

- 模块化设计
 - 分解复杂查询过程为模块化的细小工作
 - 由8个独立的模块构成



3.2 创建数据库

- SQLiteOpenHelper帮助类
 - 抽象类，具备两个抽象方法
 - onCreate(): 创建数据库
 - onUpgrade(): 升级数据库
 - 两个实例方法
 - getReadableDatabase() 和 getWritableDatabase()
 - 创建或打开数据库，并返回一个可对数据库进行读写操作的对象
 - /data/data/<package name>/databases/
 - 构造方法（4个参数）
 - 4个参数：Context上下文、数据库名、在查询数据的时候返回的自定义Cursor、当前数据库的版本号

3.2 创建数据库

1. create table Book (
2. id integer primary key autoincrement,
3. author text,
4. price real,
5. pages integer,
6. name text)

```
class MyDatabaseHelper(private val context: Context, name: String, version: Int) :
    SQLiteOpenHelper(context, name, null, version) {

    private val createBook = "create table Book (" +
        " id integer primary key autoincrement," +
        "author text," +
        "price real," +
        "pages integer," +
        "name text)"

    override fun onCreate(db: SQLiteDatabase) {
        db.disableWriteAheadLogging()
        db.execSQL(createBook)
        Toast.makeText(context, "Create succeeded", Toast.LENGTH_SHORT).show()
    }

    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
    }

}
```

3.2 创建数据库

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    val context = LocalContext.current
    val dbHelper = MyDatabaseHelper(context, "BookStore.db", 1)
    Column (
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text( text = "Hello $name!" )
        Button(
            onClick = {dbHelper.writableDatabase},
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.create_database))}
        Button(
            onClick = {},
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.add_data))}
        Button(
            onClick = {},
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.update_data))}
        Button(
            onClick = {},
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.delete_data))}
        Button(
            onClick = {},
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.query_data))}
    }
}
```

中国联通 | Club 2.17K/s 7:36

Hello Android!

Create Database

Add Data

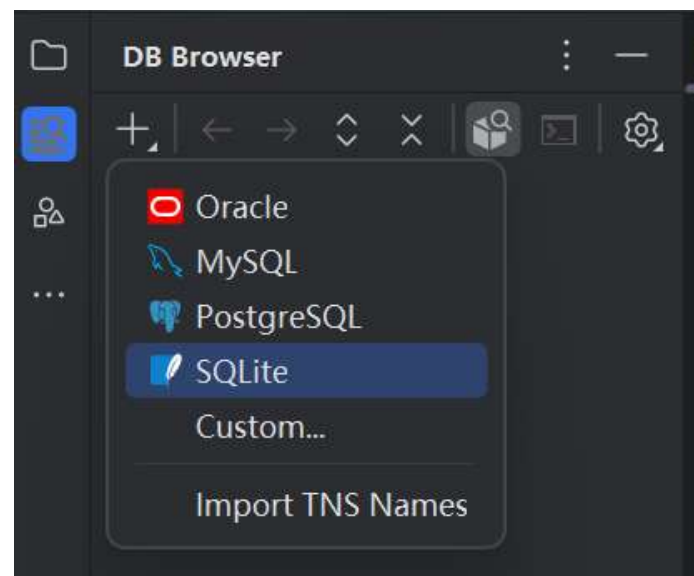
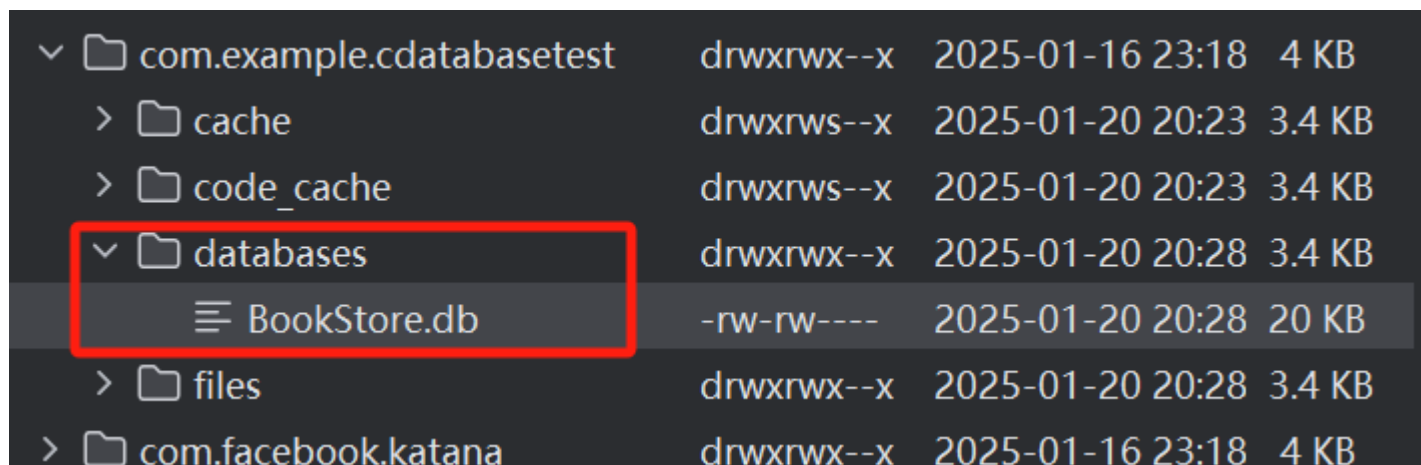
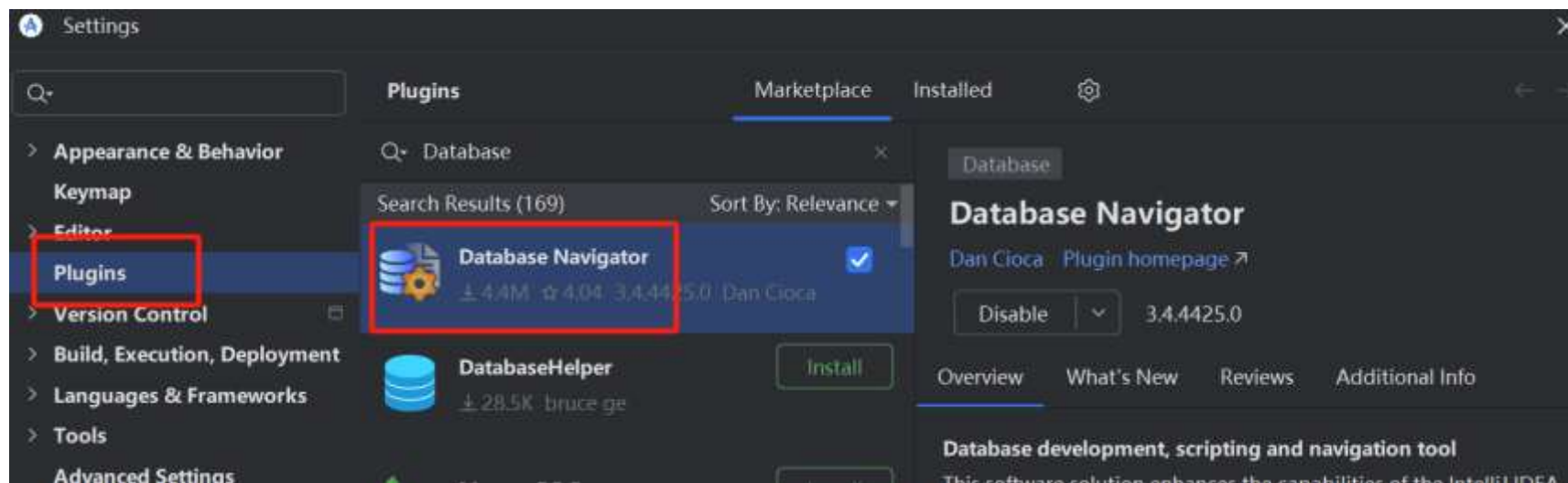
Update Data

Delete Data

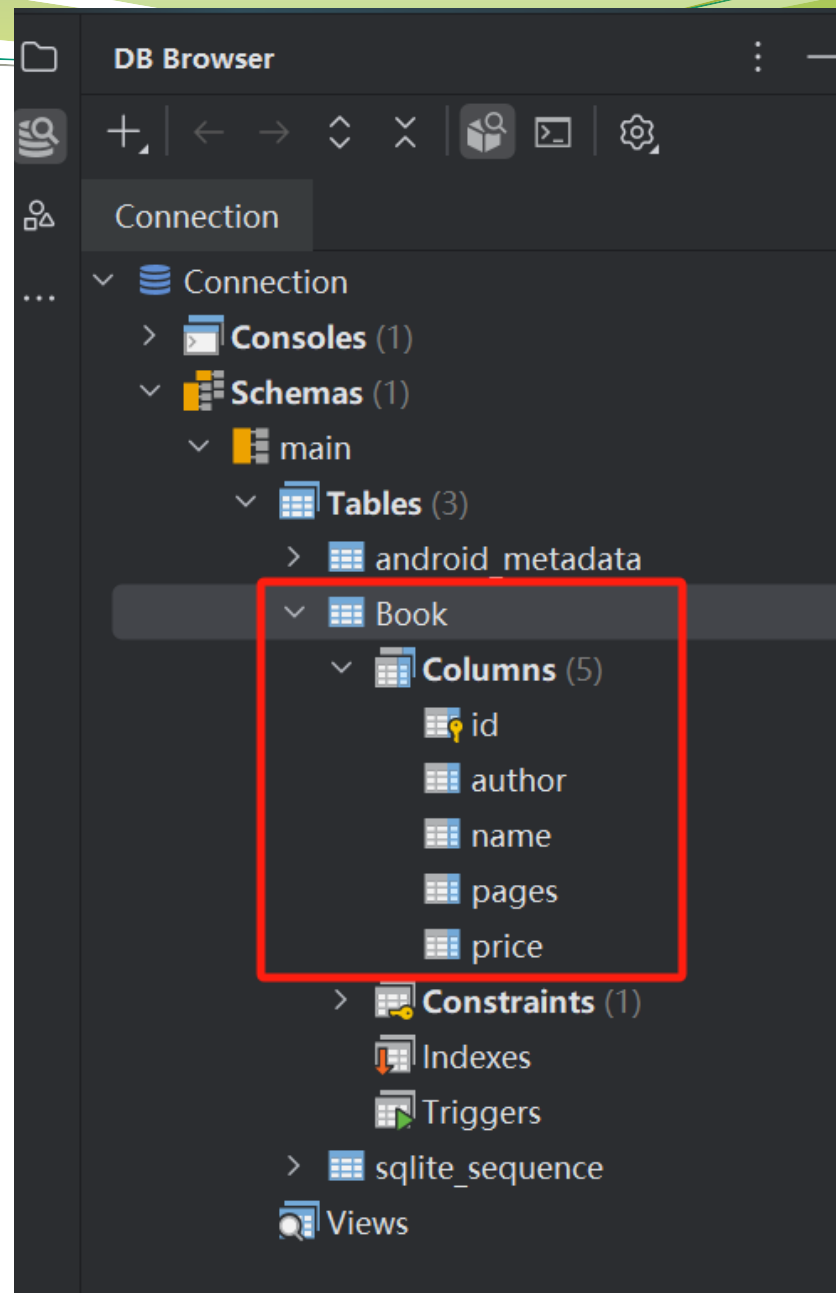
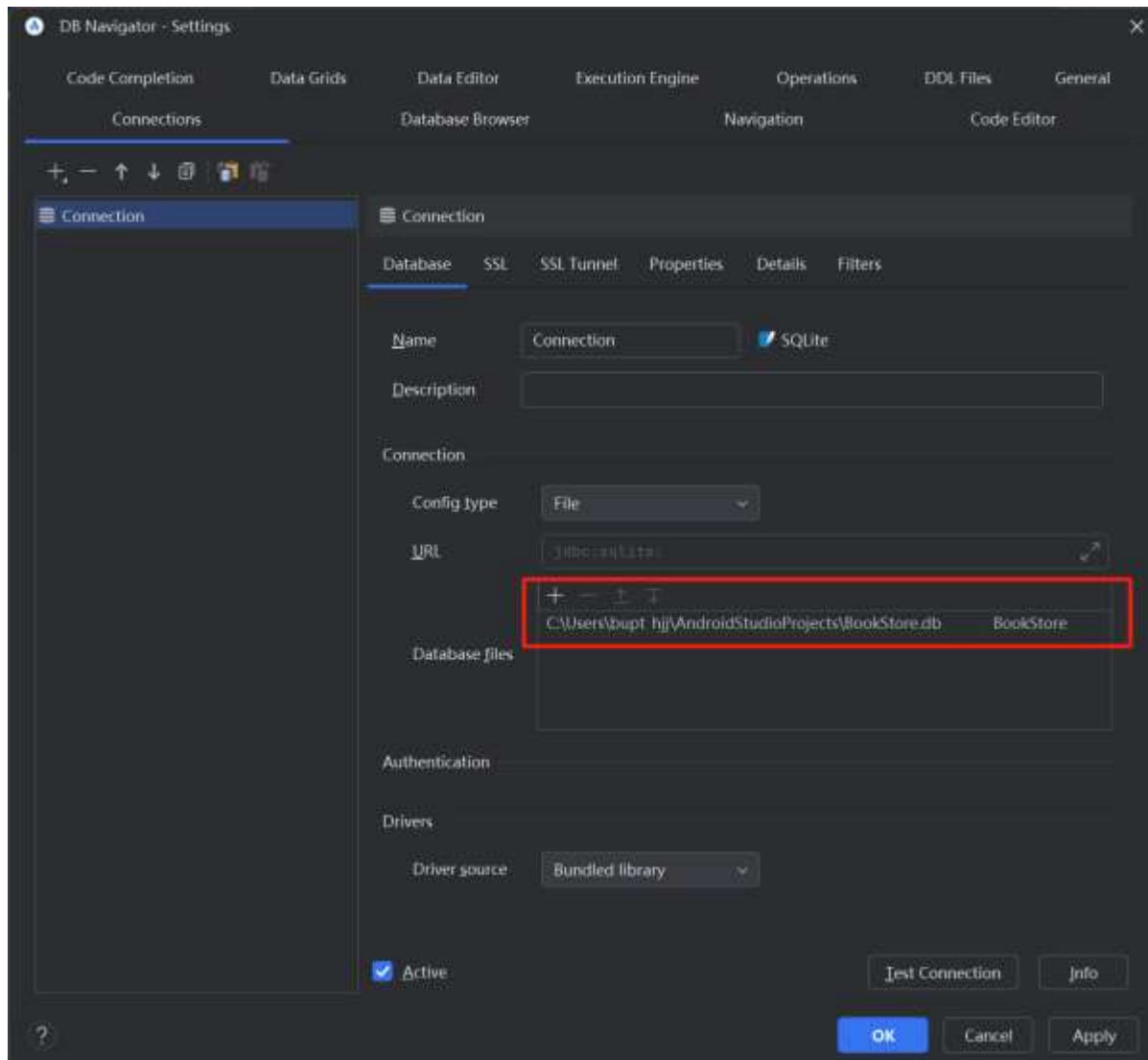
Query Data

Create succeeded

3.2 创建数据库



3.2 创建数据库



3.3 升级数据库

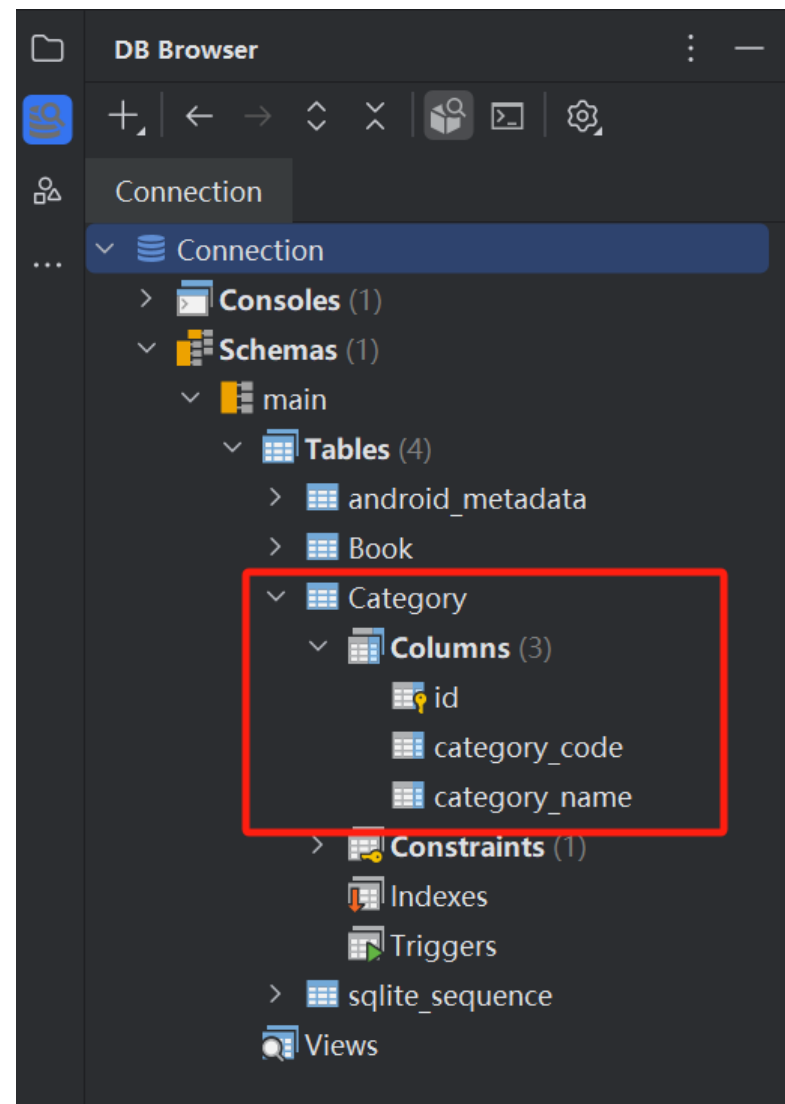
1. create table Category (
2. id integer primary key autoincrement,
3. category_name text,
4. category_code integer)

```
private val createCategory = "create table Category (" +  
    "id integer primary key autoincrement," +  
    "category_name text," +  
    "category_code integer)"  
  
override fun onCreate(db: SQLiteDatabase) {  
    db.disableWriteAheadLogging()  
    db.execSQL(createBook)  
    db.execSQL(createCategory)  
    Toast.makeText(context, text: "Create succeeded", Toast.LENGTH_SHORT).show()  
}
```


3.3 升级数据库

```
override fun onUpgrade(  
    db: SQLiteDatabase,  
    oldVersion: Int,  
    newVersion: Int) {  
    db.disableWriteAheadLogging()  
    db.execSQL("drop table if exists Book")  
    db.execSQL("drop table if exists Category")  
    db.execSQL(createBook)  
    db.execSQL(createCategory)  
    Toast.makeText(  
        context,  
        "Create succeeded",  
        Toast.LENGTH_SHORT)  
        .show()  
    }
```

```
//val dbHelper = MyDatabaseHelper(context, "BookStore.db", 1)  
val dbHelper = MyDatabaseHelper(context, "BookStore.db", 2)
```



3.4 添加数据

- 数据操作
 - CRUD——添加、查询、更新、删除
 - 辅助性方法
 - 借助SQLiteDatabase对象
- 添加数据
 - insert()
 - 参数1: 表名
 - 参数2: null
 - 参数3: ContentValues

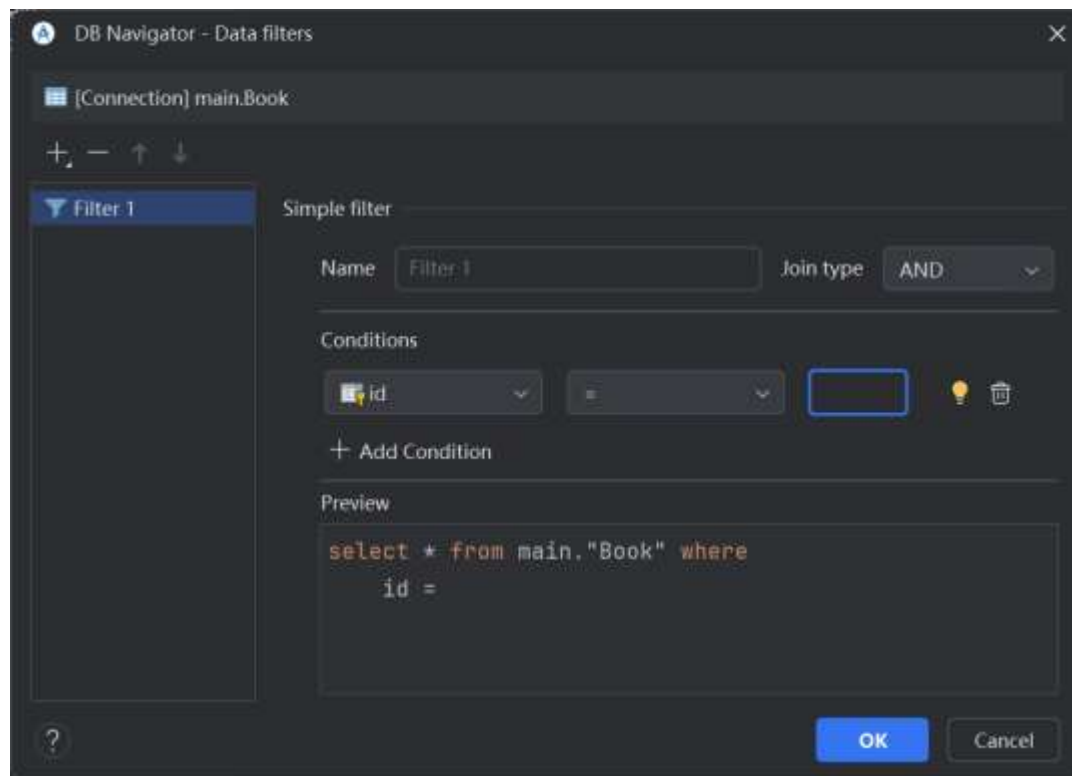
3.4 添加数据

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    .....
    Column (.....) {
        .....
        Button(
            onClick = { createDB(dbHelper) },
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.create_database))}
        Button(
            onClick = { addData(dbHelper) },
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.add_data))}
        .....
    }
}
```

```
private fun createDB(dbHelper: MyDatabaseHelper) {
    dbHelper.writableDatabase
}

private fun addData(dbHelper: MyDatabaseHelper) {
    val db = dbHelper.writableDatabase
    db.disableWriteAheadLogging()
    val values1 = ContentValues().apply {
        // 开始组装第一条数据
        put("name", "The Da Vinci Code")
        put("author", "Dan Brown")
        put("pages", 454)
        put("price", 16.96)
    }
    db.insert("Book", null, values1) // 插入第一条数据
    val values2 = ContentValues().apply {
        // 开始组装第二条数据
        put("name", "The Lost Symbol")
        put("author", "Dan Brown")
        put("pages", 510)
        put("price", 19.95)
    }
    db.insert("Book", null, values2) // 插入第二条数据
}
```


3.4 添加数据



中国联通 | Club 1728/s 9:28

Hello Android!

Create Database

Add Data

Update Data

Delete Data

Query Data

MainActivity.kt

Book

MyDatabaseHelper.kt



No Filter



Auto-Commit OFF - connected

	id	author	price	pages	name
1	1	Dan Brown	16.96	454	The Da Vinci Code
2	2	Dan Brown	19.95	510	The Lost Symbol

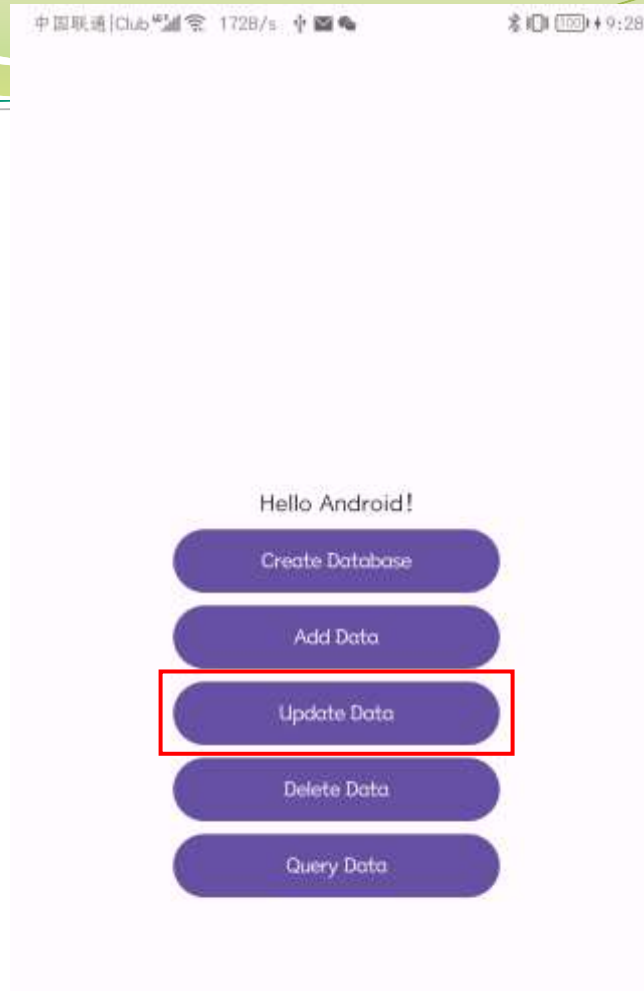
3.5 更新数据

- 更新数据
 - update()
 - 参数1: 表名
 - 参数2: ContentValues
 - 参数3、4: 指定要更新的范围

3.5 更新数据

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    .....
    Column (.....) {
        .....
        Button(
            onClick = { updateData(dbHelper) },
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.update_data))}
        .....
    }
}
```

```
private fun updateData(dbHelper: MyDatabaseHelper) {
    val db = dbHelper.writableDatabase
    db.disableWriteAheadLogging()
    val values = ContentValues()
    values.put("price", 10.99)
    db.update(
        "Book",
        values,
        "name = ?",
        arrayOf("The Da Vinci Code"))
}
```



MainActivity.kt

Book

No Filter

	id	author	price	pages	name
1	1	Dan Brown	10.99	454	The Da Vinci Code
2	2	Dan Brown	19.95	510	The Lost Symbol

3.6 删除数据

- 删除数据
 - delete()
 - 参数1: 表名
 - 参数2、3: 指定要删除的范围

3.6 删除数据

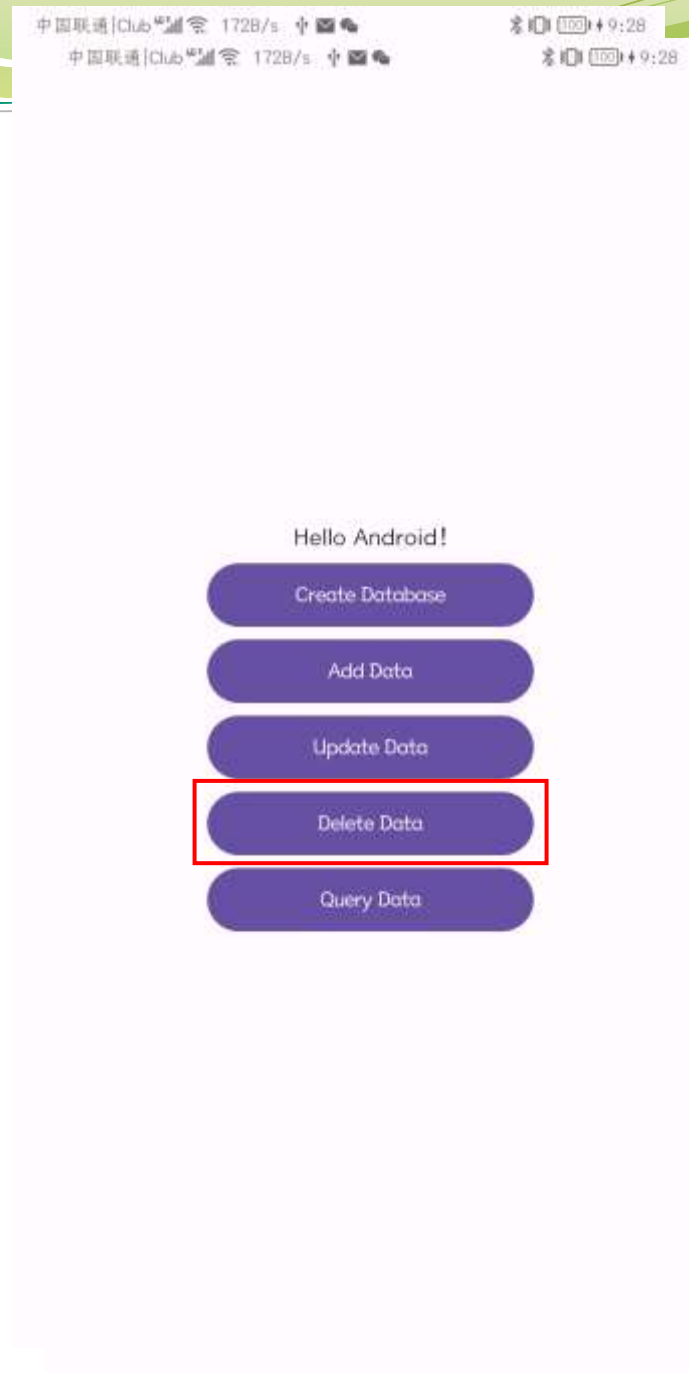
```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    .....
    Column (.....) {
        .....
        Button(
            onClick = { deleteData(dbHelper) },
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.delete_data))}
        .....
    }
}

private fun deleteData(dbHelper: MyDatabaseHelper) {
    val db = dbHelper.writableDatabase
    db.disableWriteAheadLogging()
    db.delete("Book", "pages > ?", arrayOf("500"))
}
```

MainActivity.kt Book X

🔒 🔍 📄 ⏸️ ↺ No Filter 🔥 📄 + - Auto-Commit OFF - connected

	🔑 id	author	price	pages	name
1	1	Dan Brown	10.99	454	The Da Vinci Code



3.7 查询数据

- 查询数据
 - query()

query()方法参数	对应SQL部分	描述
table	from table_name	指定查询的表名
columns	select column1, column2	指定查询的列名
selection	where column = value	指定where的约束条件
selectionArgs	-	为where中的占位符提供具体的值
groupBy	group by column	指定需要group by的列
having	having column = value	对group by后的结果进一步约束
orderBy	order by column1, column2	指定查询结果的排序方式

3.7 查询数据

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    .....
    Column (.....) {
        .....
        Button(
            onClick = { queryData(dbHelper, context) },
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.query_data))}
        .....
    }
}

private fun queryData(dbHelper: MyDatabaseHelper, context: Context) {
    val db = dbHelper.writableDatabase
    // 查询Book表中所有的数据
    val cursor = db.query("Book", null, null, null, null, null, null)
    if (cursor.moveToFirst()) {
        do {
            // 遍历Cursor对象, 取出数据并打印
            val bName = cursor.getString(cursor.getColumnIndexOrThrow("name"))
            val bAuthor = cursor.getString(cursor.getColumnIndexOrThrow("author"))
            val bPages = cursor.getInt(cursor.getColumnIndexOrThrow("pages"))
            val bPrice = cursor.getDouble(cursor.getColumnIndexOrThrow("price"))
            Toast.makeText(
                context,
                "The book titled $bName, is authored by $bAuthor, " +
                "consists of $bPages pages, and is priced at $bPrice.",
                Toast.LENGTH_LONG
            ).show()
        } while (cursor.moveToNext())
    }
    cursor.close()
}
```

中国联通 | Club 2.46K/s

10:12

Hello Android!

Create Database

Add Data

Update Data

Delete Data

Query Data

The book titled The Da Vinci Code, is authored by Dan Brown, consists of 454 pages, and is priced at 10.99.

3.8 使用SQL

1. `db.execSQL("insert into Book (name, author, pages, price) values(?, ?, ?, ?)", arrayOf("The Da Vinci Code", "Dan Brown", "454", "16.96"))`
2. `db.execSQL("insert into Book (name, author, pages, price) values(?, ?, ?, ?)", arrayOf("The Lost Symbol", "Dan Brown", "510", "19.95"))`

1. `db.execSQL("update Book set price = ? where name = ?", arrayOf("10.99", "The Da Vinci Code"))`

1. `db.execSQL("delete from Book where pages > ?", arrayOf("500"))`

1. `val cursor = db.rawQuery("select * from Book", null)`

3.9 使用事务

- 数据库事务
 - 访问并可能操作各种数据项的一个数据库操作序列
 - 操作要么全部执行, 要么全部不执行, 是不可分割的工作单位
- 在Android使用SQLite事务
 - `beginTransaction()`
 - `setTransactionSuccessful()`
 - `endTransaction()`

3.9 使用事务

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    .....
    Column (.....) {
        .....
        Button(
            onClick = { replaceData(dbHelper) },
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.replace_data))}
        .....
    }
}

private fun replaceData(dbHelper: MyDatabaseHelper) {
    val db = dbHelper.writableDatabase
    db.disableWriteAheadLogging()
    db.beginTransaction() // 开启事务
    try {
        db.delete("Book", null, null)
        throw NullPointerException()
        val values = ContentValues().apply {
            put("name", "Game of Thrones")
            put("author", "George Martin")
            put("pages", 720)
            put("price", 20.85)
        }
        db.insert("Book", null, values)
        db.setTransactionSuccessful() // 事务已经执行成功
    } catch (e: Exception) {
        e.printStackTrace()
    } finally {
        db.endTransaction() // 结束事务
    }
}
```

中国联通 | Club 32.08/s 10:28

Hello Android!

Create Database

Add Data

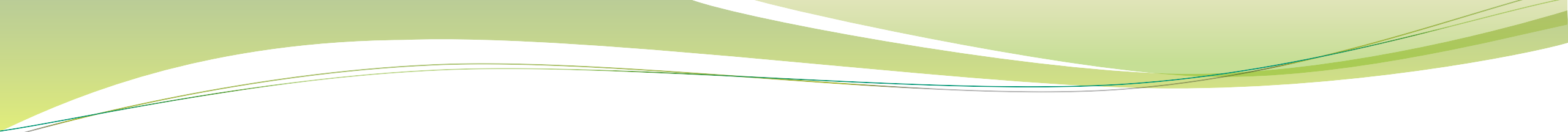
Update Data

Delete Data

Query Data

Replace Data

The book titled Game of Thrones, is authored by George Martin, consists of 720 pages, and is priced at 20.85.



感谢观看！

